UPSTACK – API HEALTH MONITORING AND INCIDENT MANAGEMENT SYSTEM

A PROJECT REPORT

Submitted by

PRANAV NARAYANAN.T

(240701393)

in partial fulfilment of the End Semester Examination for the course

CB23F35

LOW-CODE APPLICATION DEVELOPMENT USING ZOHO CREATOR

in

BACHELOR OF ENGINEERING

COMPUTER SCIENCE AND ENGINEERING



RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

Rajalakshmi Nagar, Thandalam, Tamil Nadu – 602 105

NOVEMBER 2026

RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

BONAFIDE CERTIFICATE

Certified that this project report titled "UPSTACK – API HEALTH MONITORING AND INCIDENT MANAGEMENT SYSTEM" is the bonafide work of PRANAV NARAYANAN.T (240701393), who carried out the individual project work on the Zoho Creator low-code platform for the course CB23F35 – Low-Code Application Development using Zoho Creator, during the academic year 2026–2027, Semester V, under my supervision.

SIGNATURE

Dr. MANORANJINI J
HEAD OF THE DEPARTMENT
Department of Computer Science and Engineering
Rajalakshmi Engineering College
Thandalam – 602 105

SIGNATURE

Dr. J. JINU SOPHIA

FACULTY COORDINATOR
Department of Computer Science and Engineering
Rajalakshmi Engineering College

Submitted for the End Semester Examination (Individual Project) held on _____.

INTERNAL EXAMINER

EXTERNAL EXAMINER

DECLARATION

I, PRANAV NARAYANAN.T (240701393), hereby declare that the project report titled "UPSTACK – API HEALTH MONITORING AND INCIDENT MANAGEMENT SYSTEM", submitted for the End Semester Examination of the course CB23F35 – Low-Code Application Development using Zoho Creator, is a record of original work carried out by me individually. The Zoho Creator application described in this report — including its data model, forms, reports, pages, Deluge scripts, workflows and integrations — was designed and developed by me.

I further declare that this work has not been submitted, in part or in full, for the award of any other degree or course credit. Every template, sample application, AI-assisted feature (such as Zoho Zia error analysis), code snippet or external resource used in this work has been duly acknowledged in the report. An AI assistant was used as a tutor to guide the step-by-step build and to draft this report; the application was built and tested by me on my own Zoho Creator account.

Place: Chennai
Date:

SIGNATURE OF THE STUDENT
PRANAV NARAYANAN.T
240701393

ACKNOWLEDGEMENT

I express my sincere gratitude to our Chairman Thiru. S. Meganathan, Chairperson Dr. (Mrs.) Thangam Meganathan and Vice-Chairman Mr. Abhay Meganathan S for providing the infrastructure and facilities that made this work possible.

I am grateful to our Principal, Dr. S. N. Murugesan, Rajalakshmi Engineering College, for the constant encouragement extended to students to take up industry-relevant learning.

I thank Dr. Manoranjini J, Head of the Department, Department of Computer Science and Engineering, for the guidance and support provided throughout the course.

I owe my sincere thanks to my faculty coordinator, Dr. J. Jinu Sophia, for the valuable guidance, reviews and feedback that shaped this application and report.

I also acknowledge Zoho Corporation for providing access to the Zoho Creator platform and learning resources, and I thank my family and friends for their unwavering support.

PRANAV NARAYANAN.T

ABSTRACT

Modern applications depend on many APIs to talk to databases, external services and other applications. When one of these APIs becomes unavailable or starts returning errors, the whole application can be affected, yet in most small teams the endpoints are still checked by hand, failures are noticed late, and no record is kept of what failed, when, and how it was resolved. This project addresses that problem with a centralised API health monitoring and incident management application.

UpStack is built on Zoho Creator, a low-code platform chosen because a working, hosted business application could be built and tested by a single student without server setup. The application has three connected forms — API Endpoints, API Check History and Incidents — joined by lookup fields, with list reports for endpoints, check history and incidents, and a dashboard page in the UI Builder showing the total number of API endpoints, the total number of incidents and the average response time. Automation is provided by a scheduled Deluge workflow that sends HTTP requests to every active endpoint using invokeurl and stores the HTTP status code, response time, check status and error category; a form workflow that creates an incident automatically when a check is recorded as Failed or Timeout, assigns a severity, asks Zoho Zia to analyse the error and e-mails the responsible person; and a workflow that records the resolution date and time when an incident is marked Resolved.

The application was tested with real API endpoints (the OpenAI Models API, an HTTPBin endpoint that always returns HTTP 500, and the JSONPlaceholder test API) and with controlled failure scenarios. A healthy endpoint returned HTTP 200, the failing endpoint returned HTTP 500, incidents were created, the e-mail alert was received and the resolution timestamp was recorded. The report records these results and the limitations honestly, including that monitoring depends on the configured schedule, that role-based access and multi-environment deployment were not implemented, and that no outside user acceptance feedback has been collected yet.

Keywords: Low-Code Development; Zoho Creator; Deluge Scripting; API Monitoring; Incident Management; REST API Integration; Zia AI; Workflow Automation

PROJECT AT A GLANCE

Attribute	Details
Project Title	UpStack – API Health Monitoring and Incident Management System
Application Domain	IT operations – API availability monitoring, health-check history, incident tracking and alerting
Student Name & Register No.	PRANAV NARAYANAN.T – 240701393
Zoho Creator Edition	15-day free trial of a personal account
Application Name (Link Name)	UpStack
Published / Live URL	https://github.com/pranav1393-t/LOW-CODE-ASSIGMENTS/blob/main/Upstack.ds
Demo Credentials (per role)	Single user – Administrator (account owner). Sign in with the Zoho account of the application owner (pranavnarayanan.t.2024.cse@rajalakshmi.edu.in).
Number of Forms / Subforms	3 / 0 (API Endpoints, API Check History, Incidents)
Number of Reports (by type)	List: 6 Kanban: 0 Calendar: 0 Pivot/Chart: 1 (chart on the dashboard page) Others: 0
Number of Pages / Dashboards	1 (UpStack Dashboard)
Number of Workflows	Form: 2 Schedule: 1 Approval: 0 Blueprint: 0 Batch: 0
Deluge Functions / Custom Actions	3 Deluge script actions (scheduled API health monitoring; failed-check processing with incident creation, AI analysis and e-mail; resolution timestamp); 0 stand-alone functions
Integrations	REST API (health checks on the OpenAI Models, HTTPBin and JSONPlaceholder endpoints via invokeurl; OpenAI connection) – working; Zoho Zia error analysis – working; e-mail notification – working
Roles & Profiles Defined	None beyond the default Administrator
Environments Used	Single (development / trial) environment
Date of Production Deployment	Not deployed to a separate production environment
Source Code Repository	https://github.com/pranav1393-t/LOW-CODE-ASSIGMENTS/blob/main/Upstack.ds

TABLE OF CONTENTS

ABSTRACT..... iv
PROJECT AT A GLANCE..... v
LIST OF TABLES..... viii
LIST OF FIGURES..... ix
LIST OF ABBREVIATIONS..... x
INTRODUCTION..... 1
1.1 Background..... 1
1.2 Problem Statement..... 1
1.3 Objectives..... 2
1.4 Scope of the Project..... 2
1.5 Justification for a Low-Code Approach..... 2
1.6 Organisation of the Report..... 3
LITERATURE SURVEY AND SYSTEM STUDY..... 4
2.1 Study of the Existing System..... 4
2.2 Limitations of the Existing System..... 4
2.3 Low-Code versus Traditional Development..... 4
2.4 Literature Survey..... 5
2.5 Emerging Trends in Low-Code Development..... 6
2.6 Proposed System..... 6
REQUIREMENT ANALYSIS..... 7
3.1 Stakeholders and User Roles..... 7
3.2 Functional Requirements..... 7
3.3 Non-Functional Requirements..... 8
3.4 Hardware and Software Requirements..... 8
3.5 Use Case Model..... 9
SYSTEM DESIGN..... 10
4.1 System Architecture..... 10
4.2 Data Model..... 11
4.3 Form Design..... 11
4.4 Process and Workflow Design..... 12
4.5 User Interface and Navigation Design..... 12
4.6 Access Control Design..... 13
IMPLEMENTATION..... 14
5.1 Application Setup..... 14
5.2 Forms and Data Capture..... 14
5.3 Reports..... 15
5.4 Pages and Dashboards..... 16

5.5 Deluge Scripting....16
5.6 Workflows and Automation....17
5.7 Integrations....18
5.8 User Management and Role-Based Access....19
5.9 AI and Analytics Features....19
5.10 Pro-Code Extension....19
5.11 Mobile and Portal Access....19
TESTING AND DEBUGGING....20
6.1 Testing Strategy....20
6.2 Test Cases....20
6.3 Debugging Deluge Scripts....21
6.4 Defect Log....21
6.5 User Acceptance Testing....22
DEPLOYMENT, SECURITY AND GOVERNANCE....23
7.1 Environments and Deployment Strategy....23
7.2 Performance Optimisation....23
7.3 Security Features....24
7.4 Governance and Administration....24
RESULTS AND DISCUSSION....25
8.1 Application Output....25
8.2 Achievement of Objectives....26
8.3 Comparison with the Existing System....26
8.4 Limitations of the Low-Code Approach....27
8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum....27
CONCLUSION AND FUTURE ENHANCEMENTS....28
9.1 Conclusion....28
9.2 Future Enhancements....28
9.3 Learning Outcomes and Career Relevance....28
REFERENCES....29
APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST....30
APPENDIX B – COURSE OUTCOME MAPPING....32
APPENDIX C – COMPLETE DELUGE SOURCE CODE....33
APPENDIX D – USER MANUAL....35
D.1 Accessing the Application....35
D.2 Instructions for the Administrator / Owner....35
APPENDIX E – EVALUATION RUBRIC....36
APPENDIX F – VIVA-VOCE RECORD SHEET....38

LIST OF TABLES

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed Application....4
Table 2.2: Summary of Literature Survey....5
Table 3.1: Stakeholders and User Roles....7
Table 3.2: Functional Requirements....7
Table 3.3: Non-Functional Requirements....8
Table 3.4: Hardware and Software Requirements....8
Table 3.5: Use Case Description – Process a Failed API Check....9
Table 4.1: Entities and Relationships....11
Table 4.2: Field Specification – API Check History Form....11
Table 4.3: Role–Permission Matrix (A = Add, E = Edit, V = View, D = Delete)....13
Table 5.1: Reports Implemented....15
Table 5.2: Deluge Scripts Implemented....17
Table 5.3: Workflows Implemented....17
Table 5.4: Integrations Implemented....18
Table 6.1: Test Cases....20
Table 6.2: Defect Log....21
Table 6.3: User Acceptance Feedback....22
Table 7.1: Deployment Checklist....23
Table 7.2: Performance Optimisation Results....23
Table 7.3: Security Controls Implemented....24
Table 8.1: Mapping of Objectives to Outcomes....26
Table 8.2: Existing System versus Proposed Application....26
Table A.1: Feature Coverage Checklist....30
Table B.1: Mapping of Course Outcomes to Report Chapters....32
Table E.1: Evaluation Rubric for the Individual Project....36
Table E.2: Performance Level Descriptors....36
Table F.1: Viva-Voce Questions and Responses....38
Table F.2: Marks Summary....38

LIST OF FIGURES

Figure 2.1: As-Is Process Flow of the Existing System....4
Figure 3.1: Use Case Diagram of the Application....9
Figure 4.1: System Architecture of the Application....10
Figure 4.2: Entity–Relationship Diagram....11
Figure 4.3: Workflow / Process Flow for API Monitoring and Incident Handling....12
Figure 4.4: Navigation Structure of the UpStack Application....13
Figure 5.1: API Endpoints Menu and the All Api Endpoints Report....14
Figure 5.2: API Monitoring History Report....15
Figure 5.3: All Incidents Report....16
Figure 5.4: UpStack Dashboard....16
Figure 5.5: Manage Integrations – OpenAI Connection....19
Figure 6.1: Test Evidence – API Monitoring History Showing HTTP 200 and HTTP 500 Results....21
Figure 7.1: Application in Live Mode (Dashboard with Trial Banner)....23
Figure 8.1: Scenario Step 1 – Scheduled Checks Stored in the API Monitoring History....25
Figure 8.2: Scenario Step 2 – Incidents Created for Failed Checks....25
Figure 8.3: Scenario Step 3 – Dashboard Showing Endpoints, Incidents and Average Response Time....26

LIST OF ABBREVIATIONS

Abbreviation	Expansion
AI	Artificial Intelligence
API	Application Programming Interface
ER	Entity–Relationship
FR	Functional Requirement
HTTP	HyperText Transfer Protocol
ID	Identifier
JSON	JavaScript Object Notation
KPI	Key Performance Indicator
LCAP	Low-Code Application Platform
NFR	Non-Functional Requirement
RBAC	Role-Based Access Control
REST	Representational State Transfer
SMS	Short Message Service
UAT	User Acceptance Testing
UI	User Interface
URL	Uniform Resource Locator
Zia	Zoho's AI assistant

CHAPTER 1
INTRODUCTION

1.1 Background

Almost every modern application is a collection of services that talk to each other through APIs. A web store calls a payment API, a mobile app calls a back-end API, and a dashboard calls an AI service. If any one of these endpoints becomes unavailable, slow or starts returning errors, the features that depend on it stop working, and users are affected long before the development team knows about it.

For every monitored endpoint a team needs to know its URL, the HTTP method and authentication it expects, whether it is currently healthy, how long it takes to respond, what happened the last time it failed, and who has been told. In many small teams this information is kept in personal notes, ad-hoc scripts and chat messages, and endpoints are checked by opening them manually.

This project builds a software system, UpStack, that monitors configured API endpoints on a schedule, records every health check, raises an incident automatically when a check fails, explains the error using an AI service and e-mails the responsible person. The people involved are the administrator, who configures the endpoints and watches the dashboard, and the person responsible for resolving incidents. Only publicly available test endpoints and made-up incident data are used; no real customer data is stored.

1.2 Problem Statement

Many applications depend on multiple API endpoints to perform their operations. API failures may occur due to server errors, network connectivity problems, API timeouts, incorrect endpoint configurations or external service unavailability. Without a proper monitoring system, identifying these failures and responding to them takes time, and manual checking of every API is time-consuming and unreliable.

- Endpoint details such as URL, method and authentication are scattered across notes and scripts.
- There is no single view of which APIs are healthy and which have failed.
- HTTP status codes and response times are not recorded, so past behaviour cannot be reviewed.
- Failures are noticed only when a user complains.
- The responsible person is not informed automatically, and nobody tracks whether an incident is open or resolved.
- No dashboard shows the overall health of the monitored APIs.

1.3 Objectives

- To design a normalised data model and forms for API endpoints, API check history and incidents.
- To monitor multiple API endpoints automatically using a scheduled Deluge workflow and the invokeurl task.
- To identify healthy and failed API responses and record HTTP status codes, response times and error categories.
- To maintain an API health-check history that can be reviewed later.
- To create incidents automatically for failed health-check records and assign a severity.
- To analyse API errors using Zoho Zia and store the analysis in the incident record.
- To send an e-mail notification when an incident occurs.
- To track incident status and record the resolution date and time automatically.
- To display API health information through a dashboard.
- To document the build, testing and limitations of the application honestly.

1.4 Scope of the Project

In scope: a web application with three forms, list reports, one dashboard page, one scheduled workflow, two form workflows, REST API calls through invokeurl, a connection to the OpenAI API, Zoho Zia error analysis and e-mail alerts, used by a single administrator account.

Out of scope (not implemented): role-based access for several users, real-time monitoring, retry mechanisms for failed requests, custom monitoring intervals, Slack and SMS notifications, incident assignment to team members, historical performance charts and a separate production environment. These are listed under future enhancements in Chapter 9.

1.5 Justification for a Low-Code Approach

Low-code platforms let business applications be assembled visually from forms, reports and workflows, with code needed only for custom logic [1], [5]. Zoho Creator suited this project because it provides built-in hosting, scheduled workflows, e-mail sending, charts and an AI service without any server or database setup, and because the Deluge language lets small scripts such as an HTTP request with invokeurl be added where the visual tools stop [2], [3]. A single student could therefore deliver a working, demonstrable monitoring system within the 15-day trial period instead of building a scheduler, database, mailer and user interface separately.

1.6 Organisation of the Report

Chapter 2 reviews the existing system and related low-code literature. Chapter 3 presents the requirement analysis. Chapter 4 describes the system design. Chapter 5 details the implementation on Zoho Creator. Chapter 6 covers testing and debugging. Chapter 7 discusses deployment, performance optimisation, security and governance. Chapter 8 presents results and discussion, and Chapter 9 concludes the report with future enhancements.

CHAPTER 2

LITERATURE SURVEY AND SYSTEM STUDY

2.1 Study of the Existing System

In the existing way of working, a developer or support person who wants to know whether an API is alive opens its URL in a browser or a testing tool and reads the status code. The result is noted on paper or remembered. API URLs, methods and keys are kept in personal notes. When something fails, it is usually noticed because a user reports it; the cause is then investigated manually and the responsible person is informed by phone or message. No history of past checks or past failures is kept in one place.

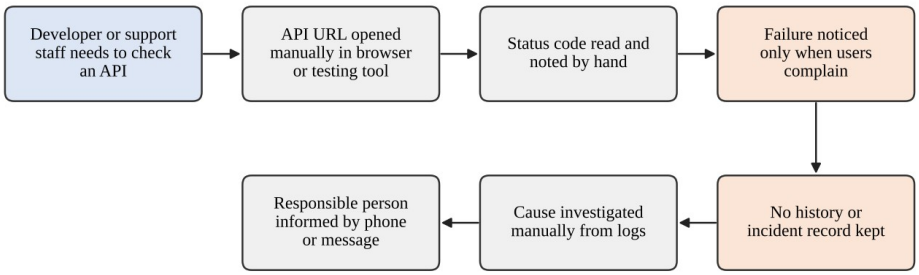


Figure 2.1: As-Is Process Flow of the Existing System

2.2 Limitations of the Existing System

- Checking every API manually is time-consuming and is not done regularly.
- No automatic detection of failures; problems are found only after users are affected.
- HTTP status codes and response times are not recorded, so trends cannot be seen.
- No incident record: nobody knows which failures are open and which are resolved.
- The responsible person is not notified automatically.
- No overview of the number of endpoints, incidents and the average response time.

2.3 Low-Code versus Traditional Development

The comparison below is a reasoned estimate for this application, not a measured result.

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed Application

Criterion	Traditional (Pro-Code) Development	Low-Code on Zoho Creator
Development time	Several weeks (scheduler, database, back-end, front-end, mailer)	About two weeks, part-time, on a trial account

Criterion	Traditional (Pro-Code) Development	Low-Code on Zoho Creator
Skills required	Programming language, framework, database, job scheduler, HTML/CSS	Form and report configuration plus basic Deluge
Hosting & infrastructure	Server or cloud hosting and a scheduler to be arranged	Hosted by Zoho; nothing to install
Scheduling & e-mail	Cron jobs and an SMTP service must be set up	Built-in schedules and sendmail
AI analysis	Separate AI service integration	Zoho Zia available inside the platform
Security & access control	Must be coded	Built-in login and permission settings
Maintenance & change	Code change, test and redeploy	Edit form or workflow in the builder
Customisation limits	Almost unlimited	Limited by platform features (monitoring runs on the configured schedule, see 8.4)
Cost	Developer time and hosting	Free trial; paid plan needed after expiry

2.4 Literature Survey

Table 2.2: Summary of Literature Survey

S.No.	Author(s) / Source & Year	Title / Product	Key Findings	Relevance to this Project
1	A. Sahay et al., 2020	Supporting the understanding and comparison of low-code development platforms	Surveys representative platforms with a conceptual comparison framework	Guides the choice of Zoho Creator for forms, workflows and integration
2	Zoho Corporation	Zoho Creator Help Documentation	Describes forms, reports, pages, workflows and connections	Primary reference for all modules built
3	Zoho Corporation	Deluge Script Reference Guide	Syntax for fetch, loops, sendmail and invokeurl	Used for the monitoring, incident and timestamp scripts
4	D. Di Ruscio et al., 2022	Low-code development and model-driven engineering: Two sides of the same coin?	Examines how low-code relates to model-driven engineering, both based on abstraction	Explains why the data model is designed before the forms
5	A. C. Bock and U. Frank, 2021	Low-Code Platform (Business & Information Systems Engineering)	Defines low-code platforms and discusses citizen development and productivity	Supports the choice of low-code for a small, fast-changing application

S.No.	Author(s) / Source & Year	Title / Product	Key Findings	Relevance to this Project
6	R. Waszkowski, 2019	Low-code platform for automating business processes in manufacturing	Shows a low-code platform automating business processes	Supports workflow automation of failure handling
7	OpenAI	OpenAI API Reference (Models endpoint)	Describes the REST endpoint that lists available models and its authentication	Real external API monitored by the application
8	HTTPBin	HTTP Request & Response Service	Provides endpoints that return a chosen status code such as 500	Used to simulate a failing API

2.5 Emerging Trends in Low-Code Development

Two trends influenced the design: citizen development, where non-programmers build working applications themselves, and AI-assisted features inside platforms. The first is reflected in the visual form, report, page and workflow builders used throughout this project; the second in the use of Zoho Zia to explain API errors. Hyper-automation — chaining small automatic actions such as scheduled checks, incident creation and e-mail alerts — is reflected in the three workflows.

2.6 Proposed System

The proposed Zoho Creator application stores endpoints, check history and incidents in linked forms, so details are entered once and found quickly. A scheduled workflow checks every active endpoint and records the result, removing the need for manual checking. A failed check creates an incident, which is analysed by AI and e-mailed to the responsible person, replacing late discovery by users. A dashboard gives the owner a single view of endpoints, incidents and response time.

CHAPTER 3

REQUIREMENT ANALYSIS

3.1 Stakeholders and User Roles

Table 3.1: Stakeholders and User Roles

Role	Description	Key Responsibilities in the Application
Administrator / Owner	Account owner who built and runs the app (only role implemented)	Manage endpoints, forms, reports and workflows; view the dashboard; manage incidents; receive e-mail alerts
Support Engineer	Person responsible for resolving API incidents (proposed, not implemented as a separate role)	Receive incident alerts, investigate the error and mark the incident Resolved
External API services	The monitored endpoints (OpenAI Models API, HTTPBin, JSONPlaceholder)	Receive health-check requests and return an HTTP status code

3.2 Functional Requirements

Table 3.2: Functional Requirements

FR ID	Requirement	Role(s)	Priority
FR-01	The system shall store API endpoint details: name, URL, provider, HTTP method, authentication details and an enable/disable flag.	Admin	High
FR-02	The system shall check active endpoints on a schedule by sending HTTP requests and record the HTTP status code and response time.	System	High
FR-03	The system shall mark each check as Healthy, Failed or Timeout and store the error message and error category.	System	High
FR-04	The system shall keep an API check history with endpoint, date, time, status code, response time, check status and error details.	Admin	High
FR-05	The system shall create an incident automatically when a check record is Failed or Timeout.	System	High
FR-06	The system shall assign a severity to each incident and maintain its status (Open / Resolved).	System	Medium
FR-07	The system shall analyse the error using Zoho Zia and store the analysis in the incident record.	System	Medium
FR-08	The system shall e-mail incident details and error information to the responsible person.	System	High
FR-09	The system shall record the resolution date and time automatically when an incident is marked Resolved.	System	Medium

FR ID	Requirement	Role(s)	Priority
FR-10	The system shall show a dashboard with total endpoints, total incidents and average response time.	Admin	Medium

3.3 Non-Functional Requirements

Table 3.3: Non-Functional Requirements

NFR ID	Category	Requirement
NFR-01	Performance	Reports and the dashboard with the test data set shall load without noticeable delay.
NFR-02	Security	Only the signed-in owner account shall access the application data; external API credentials shall be held in a platform connection, not in scripts.
NFR-03	Usability	Forms shall use dropdowns and lookups for status and endpoint selection to avoid typing errors; the left menu shall give direct access to all forms and reports.
NFR-04	Availability	The application shall be available whenever the Zoho Creator service is available (hosted by Zoho).
NFR-05	Maintainability	Endpoints, statuses and workflows shall be editable in the builder without redeploying code.
NFR-06	Reliability	A failure of one monitored endpoint shall not stop the checking of the other endpoints.

3.4 Hardware and Software Requirements

Table 3.4: Hardware and Software Requirements

Category	Specification
Platform	Zoho Creator (15-day trial, personal account)
Scripting language	Deluge (Data Enriched Language for the Universal Grid Environment)
Browser	Any modern web browser (for example Google Chrome)
Test endpoints	OpenAI Models API, HTTPBin status endpoint, JSONPlaceholder test API
AI service	Zoho Zia (error analysis); OpenAI account connection
Other Zoho services	Zoho Mail service used by sendmail / e-mail notification
Mobile	Not tested
Client hardware	Laptop or desktop computer with an internet connection

3.5 Use Case Model

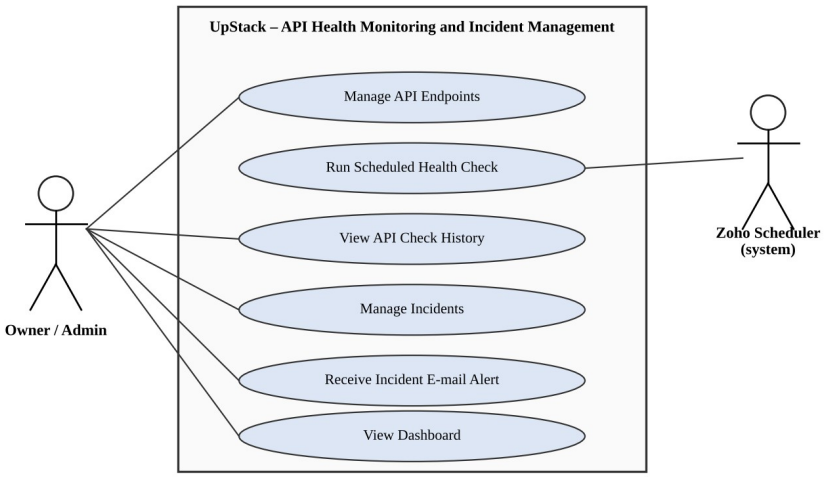


Figure 3.1: Use Case Diagram of the Application

Table 3.5: Use Case Description – Process a Failed API Check

Use Case ID	UC-01
Use Case Name	Process a failed API health check and resolve the incident
Actor(s)	Zoho Scheduler (system), Administrator / Owner
Pre-condition	At least one endpoint exists in API Endpoints and is marked active.
Main Flow	1. The daily monitoring workflow sends an HTTP request to the endpoint. 2. The status code and response time are saved as an API Check History record. 3. The result is Failed or Timeout, so the Process API Health Check workflow runs. 4. An incident is created with severity and error details; Zia analysis is stored. 5. An e-mail alert is sent to the responsible person. 6. The administrator changes the incident status to Resolved; the resolution date and time are recorded automatically.
Alternate Flow	If the check is Healthy, only the history record is saved; no incident is created and no e-mail is sent.
Post-condition	The incident appears in the incident reports with its status; the dashboard totals update.
Related FR	FR-02 to FR-09

CHAPTER 4

SYSTEM DESIGN

4.1 System Architecture

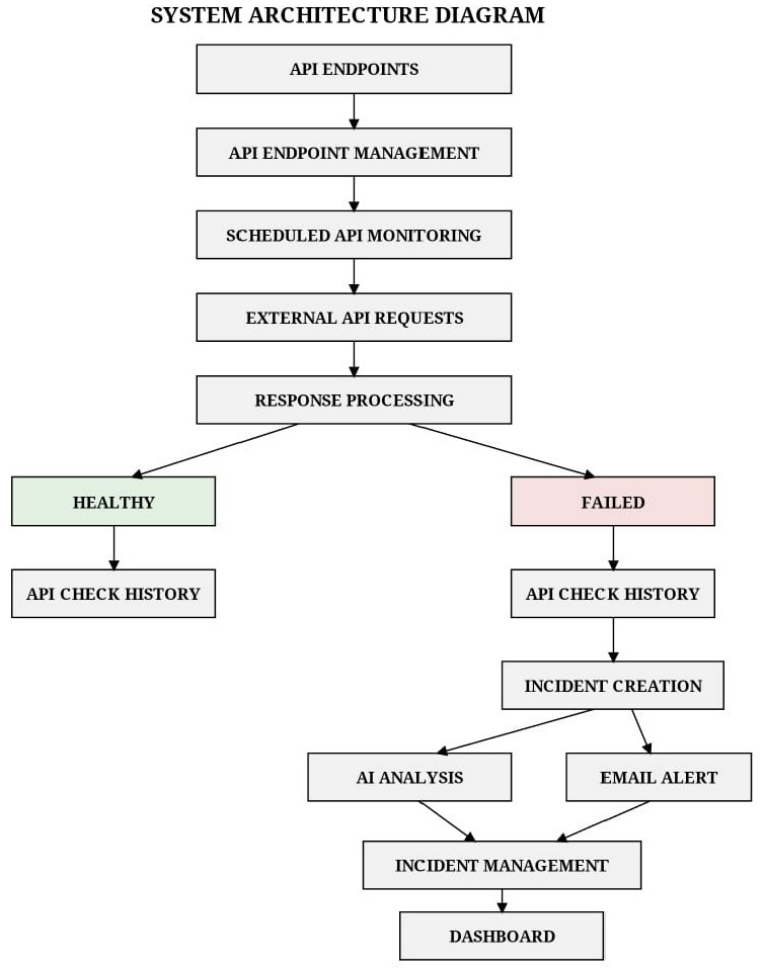


Figure 4.1: System Architecture of the Application

The system follows a centralised monitoring architecture. The configured API endpoints are managed in the API Endpoints form. The scheduled monitoring workflow sends external API requests to each active endpoint and the response is processed into a Healthy or Failed result, which is stored in API Check History. A failed record triggers incident creation; the incident is analysed by AI and an e-mail alert is sent, and both feed into incident management. The dashboard and reports present the stored data to the owner. Records are stored in Zoho's hosted data store, and the integration layer connects the app to the e-mail service, the monitored REST endpoints, the OpenAI connection and the Zia AI service.

4.2 Data Model

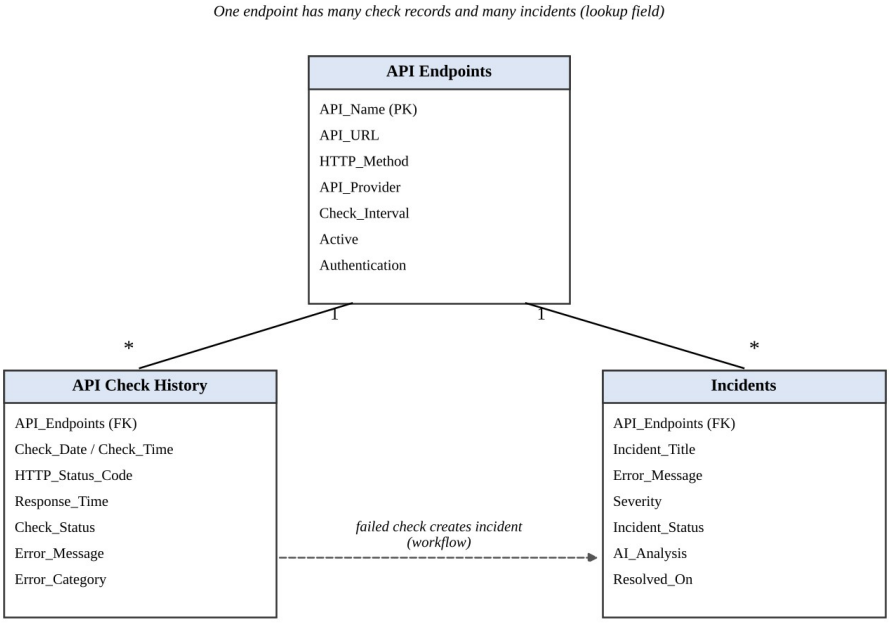


Figure 4.2: Entity–Relationship Diagram

Table 4.1: Entities and Relationships

Entity (Form)	Key Fields	Related To	Relationship Type	Implemented Using
API Endpoints	API Name, API URL, HTTP Method, API Provider, Check Interval, Active, Authentication	API Check History	One-to-Many	Lookup field "API endpoints" in API Check History
API Endpoints	as above	Incidents	One-to-Many	Lookup field "API endpoints" in Incidents
API Check History	Check Date, Check Time, HTTP Status Code, Response Time, Check Status, Error Message, Error Category	Incidents	Triggers (workflow)	Form workflow creates an incident for Failed / Timeout records

4.3 Form Design

Table 4.2: Field Specification – API Check History Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
API endpoints	API_endpoints	Lookup	Yes	—	Selects the monitored endpoint from API Endpoints
Check date	Check_date	Date	Yes	—	Date of the health check
Check Time	Check_Time	Time	Yes	—	Time of the health check

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
HTTP Status Code	HTTP_Status_ Code	Number	Yes	—	For example 200 or 500
Response Time (ms)	Response_Time	Number	Yes	—	Used for the average response time on the dashboard
Check Status	Check_Status	Drop Down	Yes	—	Healthy, Failed, Timeout; a Failed or Timeout value starts the incident workflow
Error Message	Error_Message	Multi Line	No	—	Error text returned by the check
Error Category	Error_Category	Drop Down	No	—	Category of the error (for example server error or timeout)

The API Endpoints form holds the API Name, API URL, API Provider, HTTP Method, authentication details, the Check Interval and an Active flag that enables or disables monitoring of the endpoint. The Incidents form holds an API endpoints lookup, an Incident Title, the Error Message, a Severity, the Incident Status (Open or Resolved), the AI-generated analysis and the Resolved date and time that is filled automatically.

4.4 Process and Workflow Design

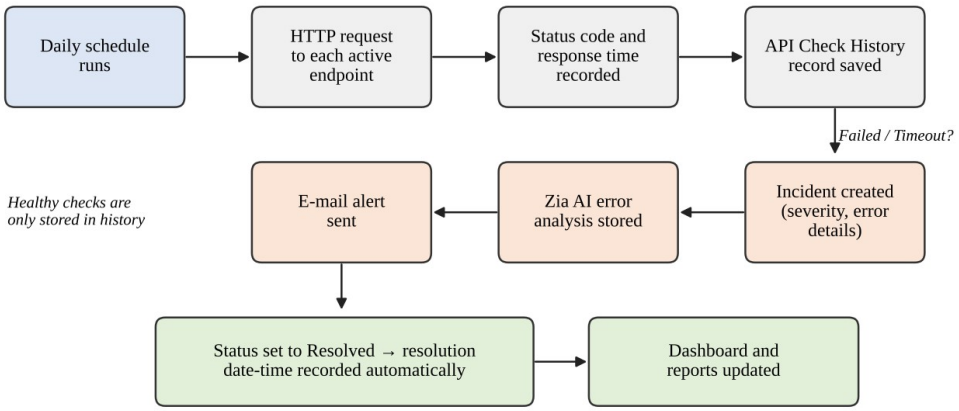


Figure 4.3: Workflow / Process Flow for API Monitoring and Incident Handling

4.5 User Interface and Navigation Design

The left menu of the application groups the forms and reports into four sections and one page: API Endpoints, App Preferences, API Check History, Incidents and the UpStack Dashboard. Each of the three sections contains the data-entry form and its list reports.

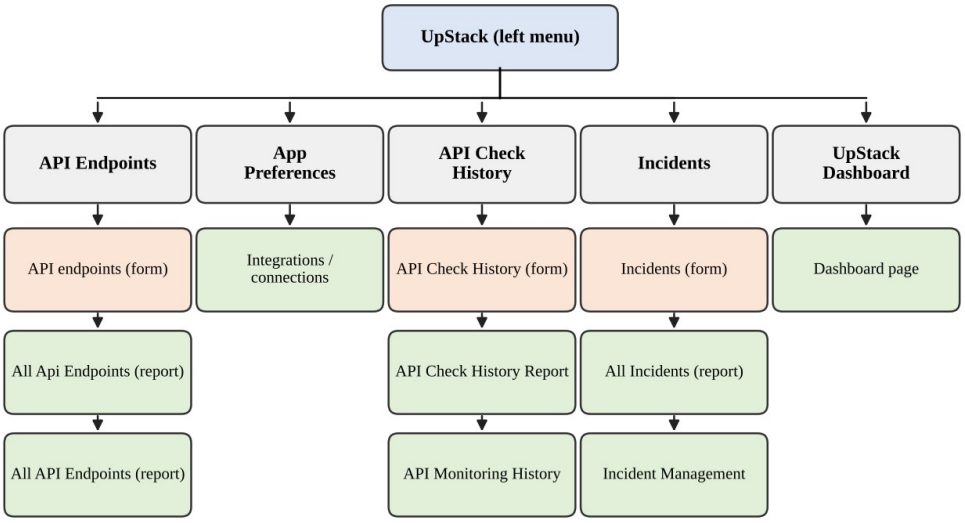


Figure 4.4: Navigation Structure of the UpStack Application

4.6 Access Control Design

Only the Administrator role is implemented. The matrix below shows the current state and the roles proposed for future work.

Table 4.3: Role–Permission Matrix (A = Add, E = Edit, V = View, D = Delete)

Component	Administrator (implemented)	Support Engineer (proposed)	Owner view-only (proposed)
Form: API Endpoints	A E V D	V	V
Form: API Check History	A E V D	A E V	V
Form: Incidents	A E V D	E V	V
Reports (all)	V	V	V
Page: UpStack Dashboard	V	V	V

CHAPTER 5
IMPLEMENTATION

5.1 Application Setup

The application "UpStack" was created from scratch on a Zoho Creator trial account, and its forms, reports, page and workflows were added from the builder. The left menu lists API Endpoints (the API endpoints form and its reports), App Preferences, API Check History (the check-history form and its reports), Incidents (the incidents form and its reports) and the UpStack Dashboard. The application icon shows the short name "UPS".

Three endpoints were configured for monitoring: an HTTPBin endpoint (https://httpbin.org/status/500) that always returns HTTP 500 and is used to simulate a failing API, the OpenAI Models API (https://api.openai.com/v1/models) as a real external service, and the JSONPlaceholder test API. Each is stored with a check interval value and an Active flag so that monitoring can be switched on or off per endpoint.

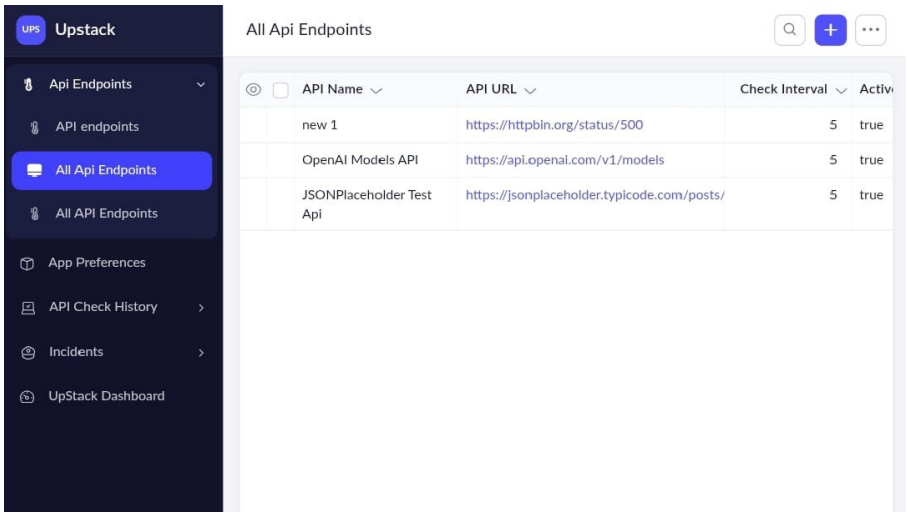


Figure 5.1: API Endpoints Menu and the All Api Endpoints Report

5.2 Forms and Data Capture

Three forms were built. In API Check History and Incidents, the "API endpoints" field is a lookup to the API Endpoints form, so an endpoint is defined once and then picked from a dropdown. API Check History captures the check date, check time, HTTP status code, response time, check status, error message and error category. Most of these records are created by the scheduled workflow, but the same form can be used to submit a record manually, which was used to test incident creation with a controlled failed record. The Incidents form stores the endpoint, an incident title, the error message, the severity, the incident status (Open or Resolved), the AI analysis and the resolution date and time.

5.3 Reports

List reports were created for the endpoints, the check history and the incidents. The API Monitoring History report shows every check in date order with the endpoint, check date, check time and HTTP status code; the report in the evidence below shows alternating HTTP 200 (OpenAI Models API) and HTTP 500 (HTTPBin endpoint) results for consecutive days, which confirms that the scheduled workflow runs and stores its results. The All Incidents report lists each incident with the endpoint, incident title and error message.

Table 5.1: Reports Implemented

Report	Type	Source Form	Filter / Purpose
All Api Endpoints	List	API Endpoints	All configured endpoints with URL, check interval and active flag
All API Endpoints	List	API Endpoints	Second listing of the endpoints (see defect D-01)
API Check History Report	List	API Check History	All health-check records
API Monitoring History	List	API Check History	Monitoring records with endpoint, date, time and HTTP status code
All Incidents	List	Incidents	All incidents with endpoint, title, error message and severity
Incident Management	List	Incidents	Incident tracking by status and resolution

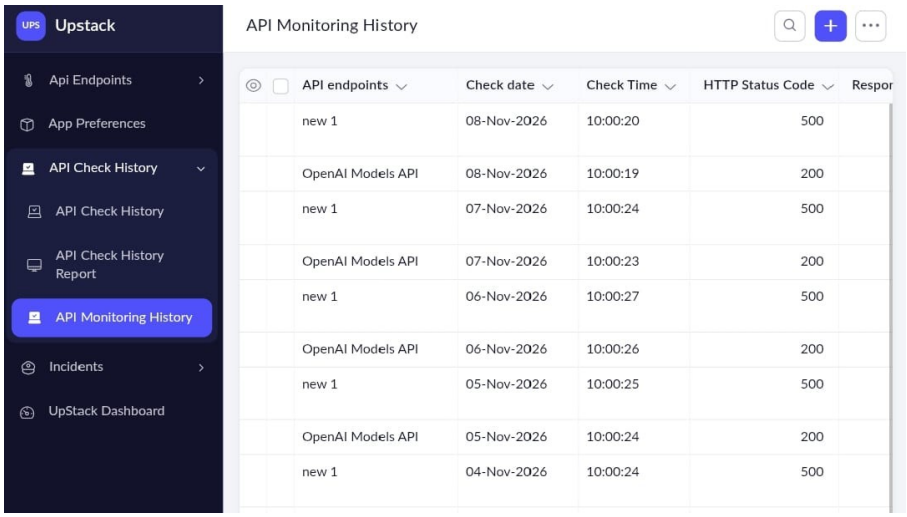


Figure 5.2: API Monitoring History Report

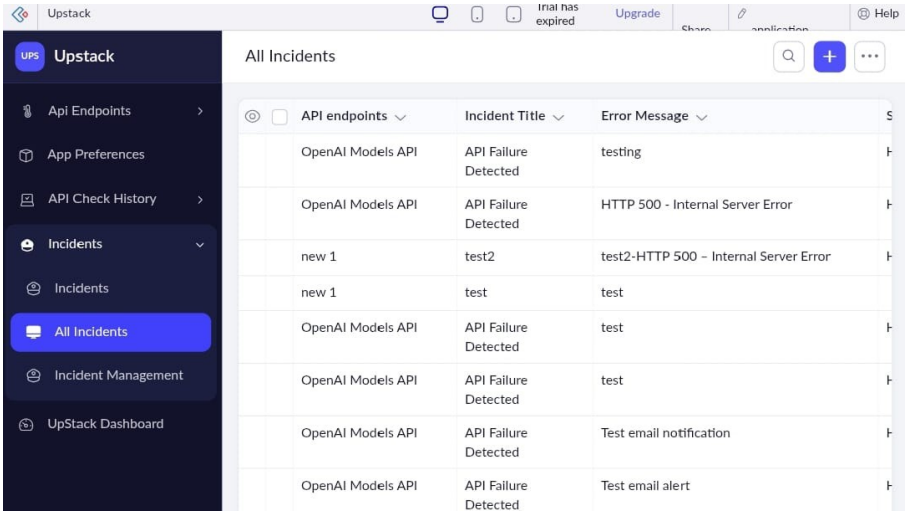


Figure 5.3: All Incidents Report

5.4 Pages and Dashboards

The page "UpStack Dashboard" was built in the Page Builder. A purple banner shows the application name "UPSTACK". Below it, summary tiles display the Total API Endpoints (3), the Total Incidents (14) and the Average Response Time in milliseconds (772.90), and a chart is drawn from the check-history records. The dashboard gives the owner one view of how many endpoints are monitored, how many incidents have been raised and how fast the monitored APIs respond.

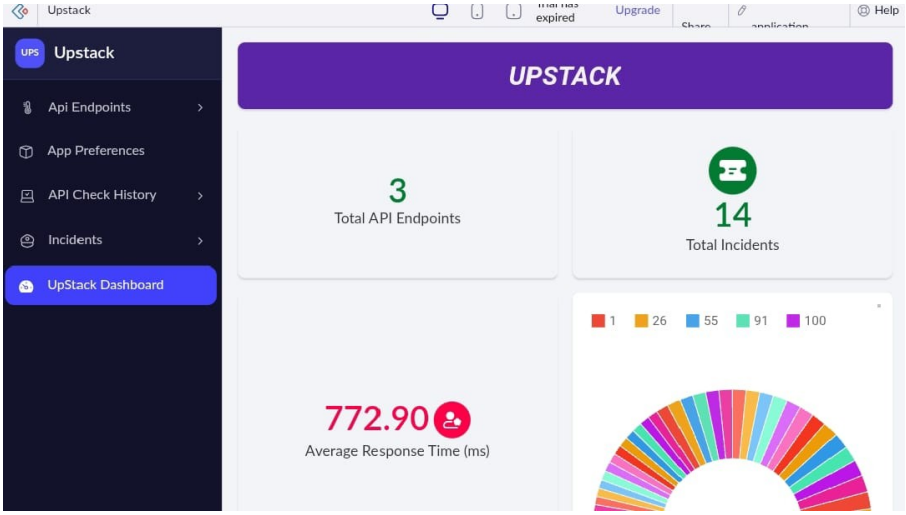


Figure 5.4: UpStack Dashboard

5.5 Deluge Scripting

Deluge is used in three places. The scheduled script fetches the active endpoints, loops through them with for each, sends an HTTP request to each with invokeurl and inserts a check-history record. The failed-check script runs when a history record is submitted and, if the check is Failed or Timeout, inserts an incident, stores the AI analysis and sends an e-

mail. The timestamp script runs when an incident is edited and records the resolution date and time. Full code is in Appendix C.

Table 5.2: Deluge Scripts Implemented

Script ID	Location / Trigger	Purpose	Deluge Concepts Used
DS-01	Schedule: Daily API Health Monitoring (daily)	Check every active endpoint and store status code, response time and check status	fetch with criteria, for each, invokeurl, try–catch, if–else, insert into
DS-02	Form workflow: Process API Health Check (API Check History, on create, success)	Create an incident for Failed / Timeout checks, store AI analysis and e-mail the responsible person	if, insert into, severity logic, Zia analysis, sendmail
DS-03	Form workflow: Automatic Incident Resolution Timestamp (Incidents, on edit, success)	Record the resolution date and time when the status becomes Resolved	if, fetch, update, zoho.currenttime

Code excerpt – DS-01 (scheduled health check)

```
activeApis = API_Endpoints[Active == true];
for each api in activeApis
{
    start = zoho.currenttime.toLong();
    try
    {
        resp = invokeurl
        [
            url :api.API_URL
            type :GET
            detailed :true
        ];
        code = resp.get("responseCode");
    }
    catch (e)
    {
        code = 0;
    }
    elapsed = zoho.currenttime.toLong() - start;
    ...
}
```

5.6 Workflows and Automation

Table 5.3: Workflows Implemented

Workflow	Type	Trigger	Condition	Action
Daily API Health Monitoring	Schedule	Runs daily at the configured time	Endpoint is Active	Deluge script DS-01 sends the request and saves a check-history record

Workflow	Type	Trigger	Condition	Action
Process API Health Check	Form workflow	API Check History created – successful form submission	Check Status is Failed or Timeout	Deluge script DS-02: create incident, assign severity, store AI analysis, send e-mail
Automatic Incident Resolution Timestamp	Form workflow	Incidents edited – successful form submission	Incident Status is Resolved	Deluge script DS-03: record resolution date and time

In plain words: the first workflow says "every day, check each active API and save the result"; the second says "when a failed check is saved, raise an incident, explain the error and send an alert"; the third says "when an incident is marked resolved, note the time". No approval, Blueprint or batch workflows were used.

5.7 Integrations

Table 5.4: Integrations Implemented

Integration	Type	Where Used	Status
Monitored REST endpoints (OpenAI Models API, HTTPBin, JSONPlaceholder)	REST API via Deluge invokeurl	Daily API Health Monitoring workflow	Working – HTTP 200 and HTTP 500 results recorded
OpenAI connection	Connection (OpenAI account) under App Preferences – Manage Integrations	Authenticated requests to the OpenAI Models API	Working
Zoho Zia	Built-in AI service	Error analysis stored in the incident record	Working
E-mail service	Notification action and sendmail	Process API Health Check workflow	Working – alert e-mail received

The OpenAI Models API is a real, authenticated service, so a Zoho Creator connection to the OpenAI account is used instead of writing the key in a script. The HTTPBin and JSONPlaceholder endpoints are public and need no key. A REST call from Deluge is the standard way to connect services that have no ready-made connector [2], [3], [7], [8].

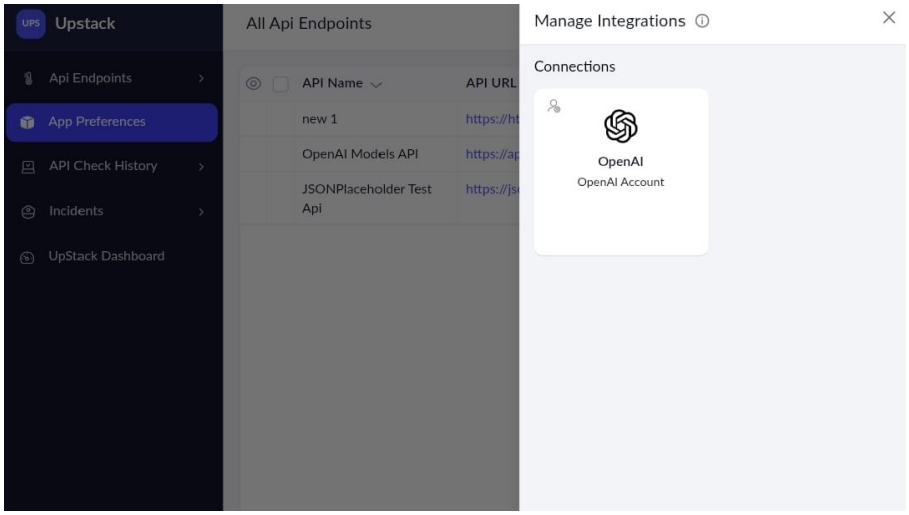


Figure 5.5: Manage Integrations – OpenAI Connection

5.8 User Management and Role-Based Access

Not implemented. The application is used by a single administrator account. Roles and permission profiles for support engineers and view-only owners are proposed in Table 4.3 and listed as future work. Reason: the project scope and the trial period were focused on monitoring, incident creation, AI analysis and e-mail alerts with one demonstration user.

5.9 AI and Analytics Features

AI: Zoho Zia is used to analyse the error information of a failed check. For example, an HTTP 500 response can be explained as a server-side failure that may require investigation. The analysis is generated automatically and stored in the incident record, so the person who receives the alert starts with a likely cause. This is a built-in Zia feature; no custom model was trained. The OpenAI account connection is used to call the OpenAI Models API as one of the monitored endpoints.

Analytics: the dashboard tiles (total endpoints, total incidents, average response time), the chart and the check-history reports summarise the data. These are rule-based analytics and are not AI.

5.10 Pro-Code Extension

Not used. No widgets, Catalyst functions or external code were required; the low-code builders and short Deluge scripts met all requirements. A possible future pro-code extension is a custom widget for response-time trend charts.

5.11 Mobile and Portal Access

Not implemented or tested. The app can be opened in a mobile browser and the Zoho Creator mobile app, but this was not verified. Role-based portals for support engineers are listed as a future enhancement.

CHAPTER 6
TESTING AND DEBUGGING

6.1 Testing Strategy

Testing was manual and functional. Real API endpoints and controlled failure scenarios were used, and the results were checked in the API Monitoring History report, the incident reports, the dashboard and the mailbox of the responsible person. The healthy case used the OpenAI Models API; the failure case used the HTTPBin endpoint https://httpbin.org/status/500, which always returns HTTP 500.

An important distinction is recorded here: the HTTP 200 and HTTP 500 results were verified through the scheduled monitoring workflow, while incident creation and the e-mail alert were verified separately by submitting a failed record through the API Check History form.

6.2 Test Cases

Table 6.1: Test Cases

TC ID	FR	Test Description	Expected Result	Actual Result	Status
TC01	FR-01	Add API endpoints in the API Endpoints form	Endpoints saved and listed in the report	Three endpoints listed (Total API Endpoints = 3)	Pass
TC02	FR-02	Healthy API: OpenAI Models API	HTTP 200	HTTP 200 – Healthy	Pass
TC03	FR-02, FR-03	Failed API: HTTPBin /status/500	HTTP 500	HTTP 500 – Failed	Pass
TC04	FR-04	Scheduled monitoring stores history	One record per endpoint per run	Records for consecutive days (04–08 Nov 2026): 200 and 500 alternate	Pass
TC05	FR-05	Submit a failed record through the API Check History form	Incident created automatically	Incident created successfully	Pass
TC06	FR-08	Check e-mail after incident creation	E-mail alert delivered	E-mail received successfully	Pass
TC07	FR-09	Change incident status to Resolved	Resolution date and time recorded	Timestamp recorded automatically	Pass
TC08	FR-10	Open the UpStack Dashboard	Totals and average response time shown	Endpoints 3, incidents 14, average response time 772.90 ms	Pass
TC09	FR-07	Check AI analysis text in an incident	Zia analysis stored in the incident	Not separately recorded in the test evidence	Pending
TC10	FR-05	Save a Healthy check record	No incident, no e-mail	Not executed during testing	Pending

6.3 Debugging Deluge Scripts

The Deluge editor reports syntax errors when a script is saved, which was the first level of checking. The scheduled workflow can be run on demand from the Deluge editor, and its effect was checked by opening the API Monitoring History report and confirming that new check records with the expected status codes appeared. The failed-check and timestamp scripts were checked through the incident reports and the received e-mail. Because the scripts depend on the link names of the forms, the field link names were checked against the form definitions when a script returned no record.

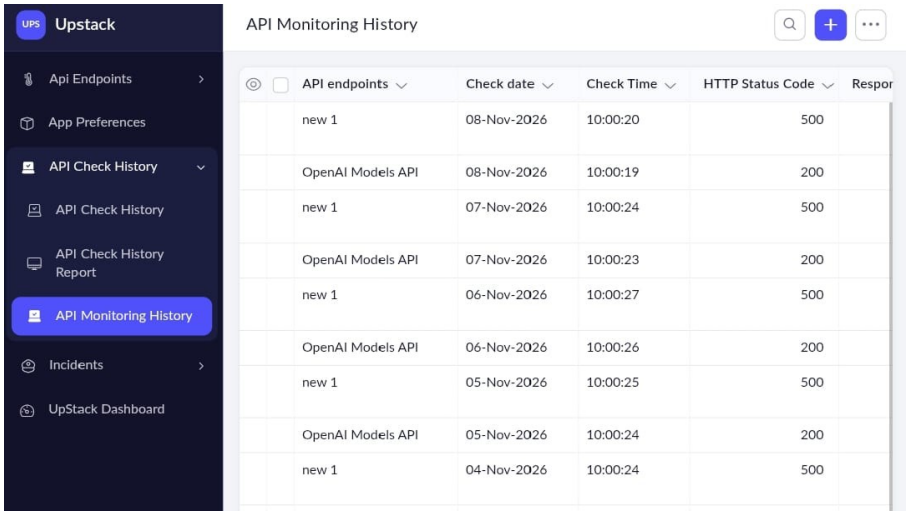


Figure 6.1: Test Evidence – API Monitoring History Showing HTTP 200 and HTTP 500 Results

6.4 Defect Log

The defects below were observed in the application and its screenshots. None of them stops the monitoring or incident workflow from working.

Table 6.2: Defect Log

ID	Description	Cause	Fix	Status
D-01	Endpoint report appears twice in the menu ("All Api Endpoints" and "All API Endpoints")	A duplicate report was created in the builder	Delete or rename one of the reports	Open
D-02	Test incidents with titles such as "test" and "testing" remain in the Incidents report	Created while testing incident creation and e-mail alerts	Delete test records before go-live	Open
D-03	The failing HTTPBin endpoint still has the default name "new 1"	Test endpoint was saved with a placeholder name	Rename it, for example "HTTPBin 500 Test"	Open

ID	Description	Cause	Fix	Status
D-04	Check Interval is stored as 5, but monitoring records appear once per day per endpoint	The schedule workflow runs on its own schedule and is not driven by the interval field	Add custom monitoring intervals (future enhancement)	Open
D-05	Trial banner shows that the trial has expired	The 15-day trial period ended	Upgrade to a paid plan to keep the application running	Open

6.5 User Acceptance Testing

No outside users have tested the application yet. The table is left for the feedback collected at the demonstration.

Table 6.3: User Acceptance Feedback

Tester Role	Task Tried	Feedback	Rating (1–5)
Not yet collected	—	No outside user has tested the application	—

CHAPTER 7

DEPLOYMENT, SECURITY AND GOVERNANCE

7.1 Environments and Deployment Strategy

The application was developed and run in a single environment on a trial account. No separate stage and production environments were created, because the application has one user and was built only for demonstration. The live URL of the application is listed in the Project at a Glance table.

Table 7.1: Deployment Checklist

Item	Done	Remarks
All forms, reports and page working	Yes	Verified in live mode
Workflows enabled	Yes	Scheduled monitoring and two form workflows
Test data removed before go-live	No	Made-up test records remain for demonstration (see D-02)
Separate stage / production environments	No	Not used

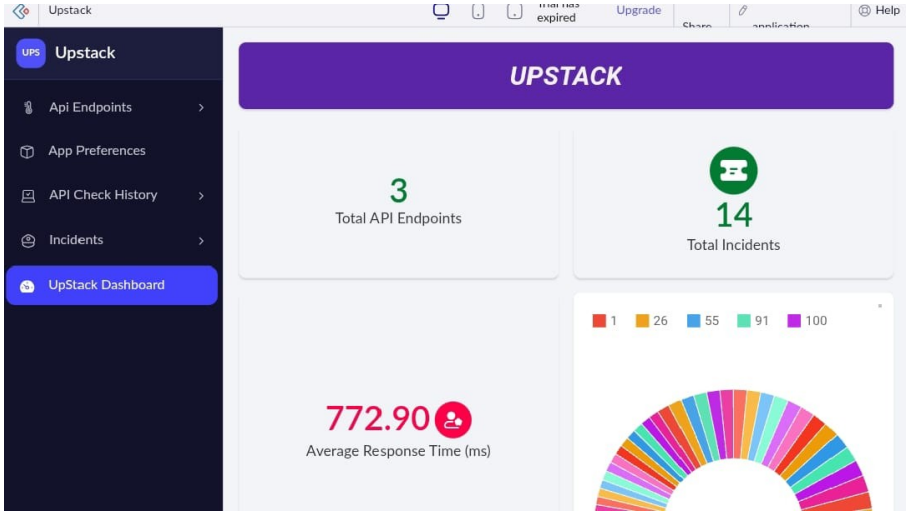


Figure 7.1: Application in Live Mode (Dashboard with Trial Banner)

7.2 Performance Optimisation

No measured performance optimisation was carried out; the test data set is small. Table 7.2 is left blank for that reason. The average response time shown on the dashboard measures the monitored APIs, not the application itself.

Table 7.2: Performance Optimisation Results

Measure	Before	After
Not applicable	—	—

7.3 Security Features

Table 7.3: Security Controls Implemented

Control	Implementation
Authentication	Access requires signing in with the owner's Zoho account
Connection for the OpenAI API	Credentials are held in a Zoho Creator connection (Manage Integrations); no API key is written in a script
Public test APIs	HTTPBin and JSONPlaceholder are public and need no key; no personal data is sent to them
Incident data	Only technical error information is stored; no customer data is stored
Role-based access	Not implemented

7.4 Governance and Administration

Administration was limited to the single owner account. Zoho Creator provides audit and user-management facilities, but they were not configured or demonstrated in this project.

CHAPTER 8

RESULTS AND DISCUSSION

8.1 Application Output

A typical scenario: the daily workflow checks the endpoints. The OpenAI Models API returns HTTP 200 and is stored as Healthy; the HTTPBin endpoint returns HTTP 500 and is stored as Failed. The failed record creates an incident, the Zia analysis is stored, and an e-mail alert is sent. When the problem has been dealt with, the administrator marks the incident Resolved and the resolution time is recorded. The dashboard totals update.

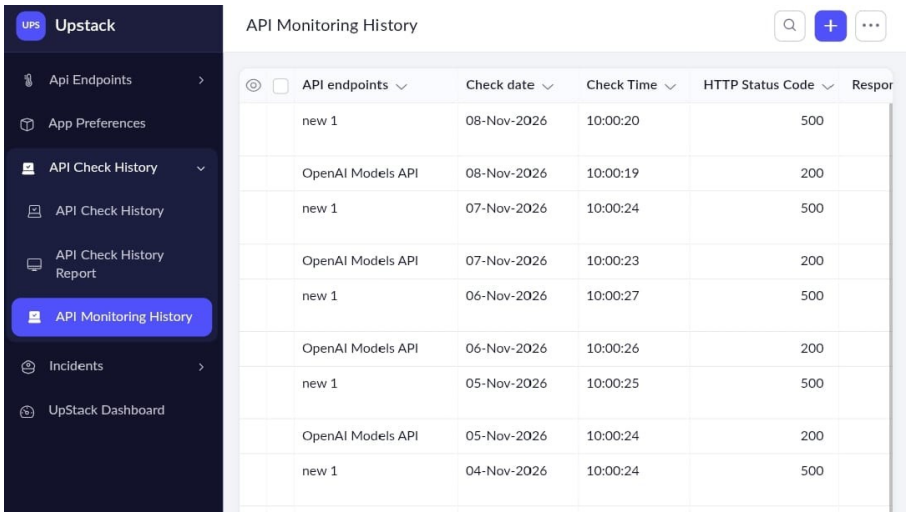


Figure 8.1: Scenario Step 1 – Scheduled Checks Stored in the API Monitoring History

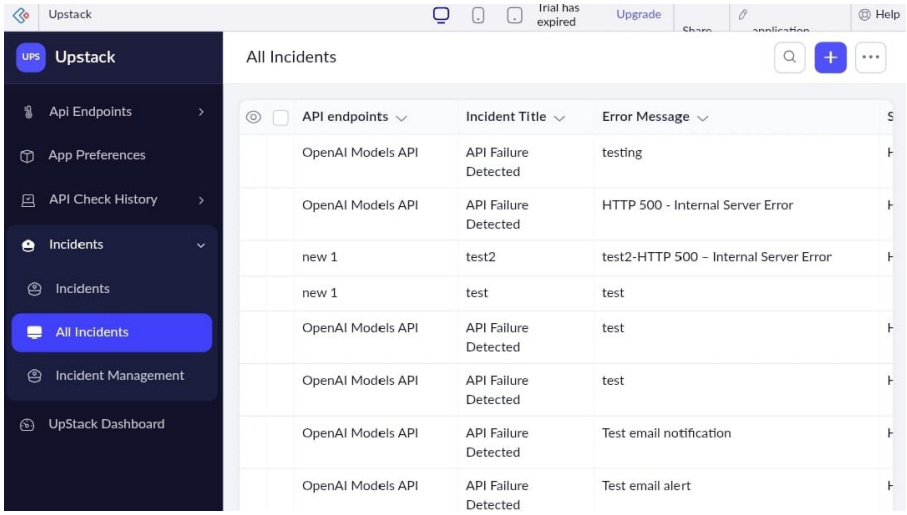


Figure 8.2: Scenario Step 2 – Incidents Created for Failed Checks

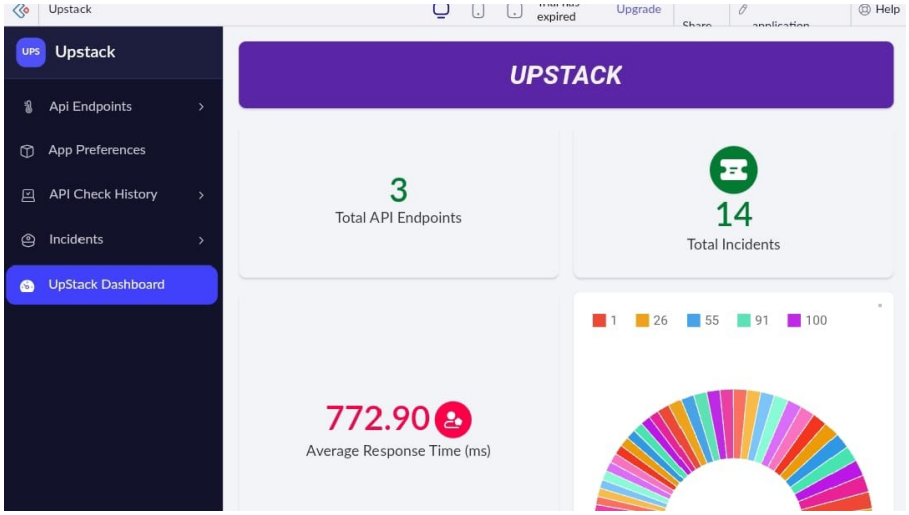


Figure 8.3: Scenario Step 3 – Dashboard Showing Endpoints, Incidents and Average Response Time

8.2 Achievement of Objectives

Table 8.1: Mapping of Objectives to Outcomes

Objective	Module / Section	Outcome
Data model and forms	4.2, 4.3, 5.2	Achieved – three linked forms
Monitor multiple endpoints on a schedule	5.5, 5.6	Achieved – HTTP 200 and 500 results recorded daily
Record status codes, response times and history	5.2, 5.3	Achieved – API Check History
Automatic incident creation with severity	5.5, 5.6	Achieved – tested through a failed-check record
AI analysis of errors (Zoho Zia)	5.9	Achieved – analysis stored in the incident
E-mail notification on incidents	5.6, 5.7	Achieved – alert e-mail received
Resolution timestamp	5.6	Achieved – recorded automatically
Dashboard	5.4	Achieved – totals, average response time and chart
Honest documentation	Ch. 6, 8	Achieved

8.3 Comparison with the Existing System

Table 8.2: Existing System versus Proposed Application

Aspect	Existing System	Proposed Application
Checking APIs	Manual, occasional	Scheduled and automatic
Record keeping	Notes and memory	Linked digital forms and history
Failure detection	Noticed when users complain	Detected at the next scheduled check

Aspect	Existing System	Proposed Application
Incident tracking	None	Automatic incidents with severity and status
Error explanation	Investigated manually	AI analysis stored in the incident
Notification	Phone or message	Automatic e-mail
Overall view	None	Dashboard with totals and average response time

No measured time savings were taken, so quantitative benefits are not claimed.

8.4 Limitations of the Low-Code Approach

- Monitoring depends on the configured schedule, so a failure is found only at the next run; real-time detection is not possible with this design.
- API authentication and connectivity affect monitoring results; a wrong key can be recorded as a failure of the API.
- AI analysis depends on the availability of the AI service.
- E-mail notifications depend on the availability of the e-mail service.
- The accuracy of error categorisation depends on the error information available.
- The trial period limits how long the application remains usable without a paid plan.

8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum

Most of the application was built with no-code tools (forms, lookups, reports, the dashboard page and e-mail actions). Low-code Deluge was needed for the scheduled HTTP requests, the incident logic and the timestamp. A pro-code extension was not needed. The experience showed that most monitoring needs fit the no-code and low-code parts, while limits such as scheduled-only checking are the points where a pro-code or microservice-based monitor would later be considered.

CHAPTER 9

CONCLUSION AND FUTURE ENHANCEMENTS

9.1 Conclusion

UpStack provides a centralised solution for monitoring API endpoints and managing API failures. The project delivered a working Zoho Creator application covering data modelling, forms, reports, a dashboard page built in the UI Builder, scheduled and form workflow automation, REST API calls, an OpenAI connection and Zia-based error analysis. Testing showed that healthy and failed responses are detected, history is stored, incidents are created, e-mail alerts are sent and resolution times are recorded. Role-based access, multi-environment deployment and outside user acceptance testing remain incomplete and are reported as such. The implementation demonstrates how low-code development and workflow automation can be used to build an effective API monitoring and incident management application.

9.2 Future Enhancements

- Real-time monitoring and faster failure detection.
- Slack and SMS notifications in addition to e-mail.
- Advanced API performance analytics and historical performance charts.
- Custom monitoring intervals per endpoint.
- Retry mechanisms for failed API requests before an incident is raised.
- Incident assignment to team members.
- Role-based access control (RBAC) for administrators, support engineers and view-only owners.
- A microservices-based monitoring architecture.
- Advanced AI-based failure prediction.
- Deployment through stage and production environments.

9.3 Learning Outcomes and Career Relevance

Through this project I learned to model data with lookups, build reports, design a dashboard, write Deluge scripts that call REST APIs with invokeurl, configure scheduled and form workflows, use connections and an AI service, and send e-mail alerts. I also learned to test systematically, separate scheduled-monitoring tests from form-based tests, and record defects. These skills apply directly to roles in business-application, low-code and IT-operations development.

REFERENCES

[1] A. Sahay, A. Indamutsa, D. Di Ruscio, and A. Pierantonio, "Supporting the understanding and comparison of low-code development platforms," in Proc. 46th Euromicro Conf. Softw. Eng. Adv. Appl. (SEAA), 2020, pp. 171–178, doi: 10.1109/SEAA51224.2020.00036.

[2] Zoho Corporation, "Zoho Creator Help Documentation." [Online]. Available: https://www.zoho.com/creator/help/ (Accessed: 2 November 2026).

[3] Zoho Corporation, "Deluge Script Reference Guide." [Online]. Available: https://www.zoho.com/deluge/ (Accessed: 2 November 2026).

[4] D. Di Ruscio, D. Kolovos, J. de Lara, A. Pierantonio, M. Tisi, and M. Wimmer, "Low-code development and model-driven engineering: Two sides of the same coin?," Software and Systems Modeling, vol. 21, no. 2, pp. 437–446, 2022.

[5] A. C. Bock and U. Frank, "Low-code platform," Business & Information Systems Engineering, vol. 63, no. 6, pp. 733–740, Dec. 2021, doi: 10.1007/s12599-021-00726-8.

[6] R. Waszkowski, "Low-code platform for automating business processes in manufacturing," IFAC-PapersOnLine, vol. 52, no. 10, pp. 376–381, 2019.

[7] OpenAI, "OpenAI API Reference – Models." [Online]. Available: https://platform.openai.com/docs/api-reference/models (Accessed: 9 November 2026).

[8] HTTPBin, "HTTP Request & Response Service." [Online]. Available: https://httpbin.org (Accessed: 9 November 2026).

[9] JSONPlaceholder, "Free fake REST API for testing and prototyping." [Online]. Available: https://jsonplaceholder.typicode.com (Accessed: 9 November 2026).

APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST

Y = implemented and working; N = not implemented (or not confirmed). Items marked Partial in the remarks were configured but not fully working and are counted as N.

Table A.1: Feature Coverage Checklist

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
I	Application created & sections/menu organised	Y	Left menu: API Endpoints, App Preferences, API Check History, Incidents, UpStack Dashboard	5.1	Fig 5.1
II	Forms with ≥ 8 distinct field types	Y	Text, URL, Number, Drop Down, Date, Time, Multi Line, Lookup, Decision Box	4.3, 5.2	Table 4.2
II	Lookup field (relationship between forms)	Y	API endpoints lookup in API Check History / Incidents	4.2, 5.2	Fig 4.2
II	Subform (parent–child data)	N	Not used	4.2, 5.2	—
II	Field validations / layout rules	N	Only mandatory fields and default values	5.2	—
II	List report with filter, grouping, custom action	Y	Endpoint, history and incident list reports (no grouping or custom action)	5.3	Fig 5.2, 5.3
II	Kanban / Calendar / Timeline / Map report	N	Not used	5.3	—
II	Pivot table / chart report	Y	Chart on the dashboard page	5.4	Fig 5.4
II	Page / dashboard built with UI Builder	Y	UpStack Dashboard	5.4	Fig 5.4
II	Deluge – form events (validate / success / field actions)	Y	Process API Health Check on form success	5.5	DS-02
II	Deluge – functions, fetch, loops, collections	Y	Daily API Health Monitoring (fetch, for each, try–catch)	5.5	DS-01
II	Email / in-app notifications	Y	Incident alert e-mail (sendmail)	5.6	DS-02
II	Form workflow	Y	Process API Health Check; Automatic Incident Resolution Timestamp	5.6	DS-02, DS-03
II	Scheduled workflow	Y	Daily API Health Monitoring	5.6	DS-01
II	Approval process	N	Not used	5.6	—
II	Blueprint	N	Not used	5.6	—
II	Batch / report workflow	N	Not used	5.6	—
II	Testing & debugging evidence	Y	Tables 6.1, 6.2	6	Fig 6.1

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
III	Integration with a Zoho service	N	No separate Zoho app (such as CRM) connected; Zia and e-mail are built-in services	5.7	—
III	Connection / REST API call (invokeurl)	Y	Health checks on OpenAI, HTTPBin and JSONPlaceholder endpoints	5.7	DS-01, Fig 5.5
III	Webhook or Creator REST API	N	Not used	5.7	—
III	Users, roles and permission profiles	N	Not implemented	5.8	—
III	Record-level access criteria / customer portal	N	Not implemented	5.8	—
III	Zia / AI feature	Y	Zia error analysis stored in incidents	5.9	DS-02
III	Analytics (charts / Zoho Analytics)	Y	Dashboard tiles, chart and reports	5.9	Fig 5.4
IV	Performance optimisation applied & measured	N	Not measured	7.2	—
IV	Deployment across environments	N	Single environment	7.1	—
IV	Security controls (encryption, OAuth, permissions)	N	Basic only: sign-in, connection for OpenAI (Partial)	7.3	—
IV	Governance (audit trail, administration)	N	Not demonstrated	7.4	—
V	Pro-code extension (Catalyst / widgets / custom functions)	N	Not used	5.10	—
V	Low-code limitations & spectrum analysis	Y	Sections 8.4, 8.5	8.4, 8.5	—
—	Mobile app / portal access	N	Not tested	5.11	—

Total features implemented: 16 / 32

APPENDIX B – COURSE OUTCOME MAPPING

Table B.1: Mapping of Course Outcomes to Report Chapters

CO	Course Outcome Statement	Evidenced In	Self-Assessed Level	Examiner Level
CO1	Explain the principles, architecture, and business value of low-code application development using Zoho Creator.	Ch. 1, 2		
CO2	Develop business applications using Zoho Creator forms, reports, workflows, and Deluge scripting.	Ch. 4, 5 (5.1–5.6), 6		
CO3	Integrate Zoho Creator applications with Zoho services and external systems using APIs, connections, and role-based security.	Ch. 4.6, 5.7–5.9		
CO4	Deploy, secure, and manage Zoho Creator applications across development and production environments following governance best practices.	Ch. 7		
CO5	Evaluate low-code development strategies, AI-assisted application development, and the transition from citizen developer to professional developer.	Ch. 5.9, 5.10, 8.4, 8.5, 9.3		

APPENDIX C – COMPLETE DELUGE SOURCE CODE

The listings below follow the logic implemented in the three workflows.

C.1 Daily API Health Monitoring (Schedule)

```
// Script ID : DS-01
// Location  : Workflow > Schedules > Daily API Health Monitoring
// Purpose   : Check every active endpoint and store the result
activeApis = API_Endpoints[Active == true];
for each api in activeApis
{
    status = "Healthy";
    errMsg = "";
    errCategory = "";
    start = zoho.currenttime.toLong();
    try
    {
        resp = invokeurl
        [
            url :api.API_URL
            type :GET
            detailed :true
        ];
        code = resp.get("responseCode");
    }
    catch (e)
    {
        code = 0;
        errMsg = e.toString();
    }
    elapsed = zoho.currenttime.toLong() - start;
    if(code == 0)
    {
        status = "Timeout";
        errCategory = "Timeout / Network";
    }
    else if(code >= 500)
    {
        status = "Failed";
        errMsg = "HTTP " + code + " - Internal Server Error";
        errCategory = "Server Error";
    }
    else if(code >= 400)
    {
        status = "Failed";
        errMsg = "HTTP " + code;
        errCategory = "Client Error";
    }
    insert into API_Check_History
    [
        Added_User = zoho.loginuser
        API_endpoints = api.ID
        Check_date = zoho.currentdate
        Check_Time = zoho.currenttime
```

```
        HTTP_Status_Code = code
        Response_Time = elapsed
        Check_Status = status
        Error_Message = errMsg
        Error_Category = errCategory
    ]
}
```

C.2 Process API Health Check (Form workflow, API Check History, on create – successful submission)

```
// Script ID : DS-02
// Location  : Workflow > Form workflows > Process API Health Check
// Purpose   : Create an incident for a Failed / Timeout check and alert by e-mail
adminMail = "[your e-mail address]";
if(input.Check_Status == "Failed" || input.Check_Status == "Timeout")
{
    severity = "Medium";
    if(input.HTTP_Status_Code >= 500)
    {
        severity = "High";
    }
    insert into Incidents
    [
        Added_User = zoho.loginuser
        API_endpoints = input.API_endpoints
        Incident_Title = "API Failure Detected"
        Error_Message = input.Error_Message
        Severity = severity
        Incident_Status = "Open"
    ]
    // AI analysis (Zoho Zia) of the error message is stored in the incident record
    sendmail
    [
        from :zoho.adminuserid
        to :adminMail
        subject :"API Incident Detected"
        message :"An API check failed. HTTP status: " + input.HTTP_Status_Code +
"<br>Error: " + input.Error_Message
    ]
}
```

C.3 Automatic Incident Resolution Timestamp (Form workflow, Incidents, on edit – successful submission)

```
// Script ID : DS-03
// Location  : Workflow > Form workflows > Automatic Incident Resolution Timestamp
// Purpose   : Record the resolution date and time when an incident is Resolved
if(input.Incident_Status == "Resolved")
{
    rec = Incidents[ID == input.ID];
    if(rec.Resolved_On == null)
    {
        rec.Resolved_On = zoho.currenttime;
    }
}
```

APPENDIX D – USER MANUAL

D.1 Accessing the Application

Open the live URL in a web browser, sign in with the owner's Zoho account and choose "UpStack Dashboard" from the left menu.

D.2 Instructions for the Administrator / Owner

1. To add an API endpoint: open API Endpoints, click the add (+) button, enter the API name, URL, HTTP method and check interval, set Active and submit.
2. To switch monitoring on or off: edit the endpoint in All Api Endpoints and change the Active flag.
3. To see monitoring results: open API Check History > API Monitoring History and read the HTTP status code of each check.
4. To see failures: open Incidents > All Incidents, or read the incident alert e-mail.
5. To close an incident: open Incident Management, edit the incident and set the status to Resolved; the resolution date and time are filled automatically.
6. To see overall health: open the UpStack Dashboard for the total endpoints, total incidents and average response time.

APPENDIX E – EVALUATION RUBRIC

(For Examiner Use Only)

Student Name	PRANAV NARAYANAN.T	**Register No.**	240701393
Project Title	UpStack – API Health Monitoring and Incident Management System	**Date**	

Table E.1: Evaluation Rubric for the Individual Project

S.No.	Criterion	Indicators	Max. Marks	Marks Awarded
1	Problem Definition & Requirement Analysis	Clarity of problem, measurable objectives, complete FR/NFR, use cases	10	
2	System Design	Architecture, ER/data model, form specifications, workflow design, permission matrix	10	
3	Forms, Reports & Pages	Variety of field types, lookups/subforms, report types, UI Builder dashboards	15	
4	Deluge Scripting & Workflows	Correctness and depth of scripts; form, schedule, approval, Blueprint, batch workflows	15	
5	Integration, RBAC & AI	Zoho/external integrations, connections, roles & permissions, Zia/analytics	10	
6	Testing, Deployment, Security & Governance	Test coverage, environments, optimisation, security controls, audit	10	
7	Live Demonstration	End-to-end working of the application across roles; feature checklist verified	15	
8	Project Report	Adherence to template and format, clarity, figures, references	5	
9	Viva-Voce	Understanding of the platform, design decisions and low-code concepts	10	
	TOTAL		**100**	

Table E.2: Performance Level Descriptors

Level	% of Max. Marks	Descriptor
Excellent	90–100	Feature implemented completely, correctly and beyond the basic requirement; student explains design choices confidently.

Level	% of Max. Marks	Descriptor
Good	70–89	Feature implemented correctly with minor gaps; explanation adequate.
Satisfactory	50–69	Feature partially implemented or works only for the main path; limited explanation.
Needs Improvement	< 50	Feature missing, non-functional or not understood by the student.

Signature of Internal Examiner Name: _____	**Signature of External Examiner** Name: _____

APPENDIX F – VIVA-VOCE RECORD SHEET

(For Examiner Use Only)

Student Name	PRANAV NARAYANAN.T	**Register No.**	240701393
Department / Section	Computer Science and Engineering / CSE	**Date of Viva**	
Application URL Verified	Yes / No	**Demo Duration**	_____ min

Table F.1: Viva-Voce Questions and Responses

Q.No.	Question / Topic Asked	Unit	Response (E / G / S / NI)	Remarks
1				
2				
3				
4				
5				
6				
7				
8				

E – Excellent G – Good S – Satisfactory NI – Needs Improvement

Table F.2: Marks Summary

Component	Max. Marks	Marks Awarded
Criteria 1–8 (Appendix E)	90	
Viva-Voce	10	
TOTAL	100	

Examiners' Remarks:

_____

_____

INTERNAL EXAMINER	EXTERNAL EXAMINER
Name: _____	Name: _____